

1 Introduction to Finite Automata

Lecture 3 split the problem of specifying a language into two dual halves – **recognizers**, machines that read a string and answer yes or no, and **generators**, devices that write out the strings of L without reading anything. That lecture, and Lecture 4 after it, worked entirely on the generator side: regular expressions, grammars, and the Chomsky hierarchy that sorts grammars by rule shape. The recognizer side is still just Lecture 3’s one-line definition – “a machine M that takes a string w and outputs ACCEPT or REJECT.” Building that machine forces one question to the front, and it is the question this whole document is really about: *what information about the past does a machine need to remember, in order to decide correctly about the future?* We answer it three times, at three levels of precision – with a turnstile, with a menu of machine types, and finally with a formal definition – and land on the simplest disciplined answer ToC uses: the **deterministic finite automaton (DFA)**. Every later machine in this course (NFA, PDA, Turing machine) asks the same question again, with a progressively richer notion of “remember.”

1.1 From a Turnstile to a State Machine

Picture the turnstile at a train station. It has exactly two modes: *locked*, blocking you, and *unlocked*, letting one person through. Drop a coin and it unlocks; push through and it locks again; push it while it is already locked and nothing happens; drop a coin while it is already unlocked and it just stays unlocked. Notice what the turnstile is *not* doing: it is not keeping a running log of every coin and every push you have ever fed it. Its entire memory, at any instant, is which of the two modes it is currently in – and that is the core idea a **state** is built on: *a state is not a record of the past. It is the information about the past that is sufficient to determine what matters for the future.* The turnstile does not need to remember every coin and push it has ever seen – only enough to know what happens next, and “Locked” or “Unlocked” is already enough. That is the turnstile’s answer to this document’s central question – one bit of memory, and not one bit more.

Finite state machine (informal)

A **finite state machine (FSM)** is a device that is, at every instant, in exactly one of a finite set of **states**. It reads input symbols one at a time; each symbol, combined with the current state, determines the next state via a **transition**. The machine keeps no other memory – everything it “remembers” about the input read so far is compressed into which state it is currently sitting in.

The turnstile is a two-state FSM. Write its states as {Locked, Unlocked} and its input symbols as {coin, push}; the whole behaviour described above is four transitions:

Locked $\xrightarrow{\text{coin}}$ Unlocked, Locked $\xrightarrow{\text{push}}$ Locked, Unlocked $\xrightarrow{\text{coin}}$ Unlocked, Unlocked $\xrightarrow{\text{push}}$ Locked.

Feed it the sequence coin, push, push, coin, push and just follow the state after each symbol: start at Locked; coin takes it to Unlocked; push takes it back to Locked; the second push changes nothing (still Locked); coin unlocks it again; the final push locks it.

Five inputs, five transitions, and at every instant you can say exactly which of two states the turnstile occupies – that running trace *is* what it means to run an FSM on an input sequence.

Exercises: Modeling with States

1. A vending machine's coin slot is jammed shut once a purchase is dispensed, and only unjams after the machine is restocked. Model it as an FSM: list its states and input symbols, and give its transitions in the same style as the turnstile above. (Hint: how many states do you actually need to remember “jammed” versus “ready”?)
2. Trace the turnstile FSM above on the input sequence `push, coin, coin, push, coin`. Which state does it end in?
3. A stapler can be either loaded or empty, and a “staple” action only does something when it is loaded (after which, if it only holds one staple, it becomes empty). Model it as an FSM.

1.2 Flavors of State Machines, and the One ToC Wants

Look back at Section [1.1](#): the turnstile FSM never produced any output beyond the state it ended in. Real state machines usually do produce output, and there are three common ways of using a finite-state machine, depending on what you attach that output to.

Three flavors of FSM

- **Moore machine.** Output is a function of the *current state* alone, produced at every step just by being in that state – e.g., a state literally labeled “door open,” always emitting “open” while the machine sits there.
- **Mealy machine.** Output is a function of the *state and the input symbol together*, produced on each transition – e.g., a buzzer that sounds exactly on the transition “push while Locked,” not while sitting in any state.
- **Acceptor (recognizer).** No output alphabet at all. Some states are simply marked **final** (or **accepting**); the machine's only “output,” produced once, after the *entire* input has been read, is whether it ended in a final state.

ToC's central question is never a running commentary on every prefix of a string – it is one yes/no answer about the string as a whole: *does w belong to L ?* That is exactly what an acceptor gives you, and it is why every recognizer this course builds – finite automata now, pushdown automata and Turing machines later – is an acceptor, never a Moore or Mealy machine: we give up per-step output for the one thing ToC actually needs, a single accept/reject decision at the end.

To think!!

Section 1.1's turnstile, as drawn, is neither Moore nor Mealy nor an acceptor – it is just a bare FSM with no output at all. Suppose you only want to ask: *after this whole sequence of coins and pushes, is the turnstile currently unlocked?* Can you turn it into an acceptor for exactly that question without changing a single state or transition already fixed in Section 1.1? (Section 1.3 answers this explicitly.)

Exercises: Naming the Flavor

1. A traffic light cycles red $\rightarrow$ green $\rightarrow$ yellow $\rightarrow$ red, and at every instant displays the color of its current state. Which of the three flavors is this, and why?
2. A parity checker reads a bit string and, on every single bit read, immediately emits whether the count of 1's seen *so far* is even or odd. Which flavor is this? (Hint: does the output depend on the state alone, or on the state *and* the bit just read?)
3. Redraw the parity checker of the previous exercise as an acceptor instead, for the single question “is the total number of 1's in the whole string even?” What has to change, and what stays the same?

1.3 The Formal Definition of a Deterministic Finite Automaton (DFA)

Section 1.1 gave states and transitions in prose – fine for one machine, not for reasoning about automata in general. That FSM was the general idea; a **deterministic finite automaton (DFA)** is the specific, disciplined member of the family where, on every state and every input symbol, *exactly one* transition applies – never zero, never more than one. That single-valuedness is what **deterministic** means, and it is this document's central question formalized: whatever the machine must remember about the past is compressed into a state in Q , and δ updates that memory one symbol at a time.

Deterministic finite automaton (DFA)

A **deterministic finite automaton (DFA)** is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$:

- Q – a finite set of **states**.
- Σ – a finite input **alphabet**.
- $\delta : Q \times \Sigma \rightarrow Q$ – the **transition function**: exactly one next state for every state-symbol pair – the single-valuedness that makes M *deterministic*.
- $q_0 \in Q$ – the **initial state**, where every run starts.
- $F \subseteq Q$ – the **final (accepting) states**. M **accepts** w if, starting at q_0 and following δ one symbol at a time, it ends in some state in F ; otherwise M **rejects** w .

Accepting or rejecting one string is only half the point – the strings M accepts together form a language, exactly Lecture 1's $L \subseteq \Sigma^*$:

$$L(M) = \{w \in \Sigma^* : M \text{ accepts } w\}.$$

$$\boxed{\text{DFA}} \longrightarrow \boxed{\text{accepts strings}} \longrightarrow \boxed{\text{recognizes a language}}$$

Every later question in this module – is L regular, can two DFAs recognize the same L , how do we build one for a given L – is a question about this chain.

δ can be written down two ways, and both carry the same information:

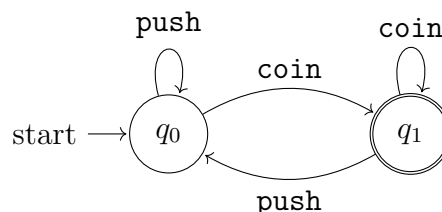
Two notations, one function

A **transition table** lists states as rows, input symbols as columns, $\delta(q, a)$ as the entry – the format algorithms read directly. A **transition diagram** draws states as circles, q_0 marked with a bare incoming arrow, F as double circles, and each $\delta(q, a) = q'$ as an arrow $q \rightarrow q'$ labeled a – the format a human reads directly. Neither is more “correct”; both are the same δ .

Now formalize the turnstile as a DFA, answering Section 1.2's “To think!!” box: keep every state and transition from Section 1.1, rename Locked/Unlocked as q_0, q_1 , and set $F = \{q_1\}$ – “accept” means “unlocked once the input runs out.”

$$Q = \{q_0, q_1\}, \quad \Sigma = \{\text{coin}, \text{push}\}, \quad q_0 \text{ (initial)}, \quad F = \{q_1\}.$$

δ	coin	push
q_0	q_1	q_0
q_1	q_1	q_0



Trace $w = \text{coin push coin}$: $q_0 \xrightarrow{\text{coin}} q_1 \xrightarrow{\text{push}} q_0 \xrightarrow{\text{coin}} q_1 \in F$ – **accept**. Trace $w' = \text{coin push push}$: $q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_0 \notin F$ – **reject**. Same δ – only the input changed, and that alone flipped the answer.

One trace matters on principle: the empty string ε ($|w| = 0$, Lecture 1). M reads no symbols, so it never leaves q_0 – ε is accepted exactly when $q_0 \in F$. Here $q_0 = \text{Locked} \notin F$, so M rejects ε : an untouched turnstile is still locked.

Exercises: Building a Formal DFA

1. A gate needs *two coins in a row* before it unlocks; it remembers the count with three states – zero, one, two coins seen (locked, locked, unlocked). From zero: **coin** $\rightarrow$ one; **push** does nothing. From one: **coin** $\rightarrow$ two (unlocks); **push** $\rightarrow$ zero. From two (unlocked): **push** $\rightarrow$ zero (visitor passes through); **coin** is wasted, stays at two. Give its formal 5-tuple $(Q, \Sigma, \delta, q_0, F)$, its transition table (all six entries), and its transition diagram.
2. Trace your automaton on $w = \text{coin coin push}$ and on $w' = \text{coin push coin coin}$. Which does it accept? Does it accept ε ?
3. Compare your gate (M_2) against this section’s turnstile (M_1): both read $\Sigma = \{\text{coin}, \text{push}\}$, so both recognize a language $L(M_1), L(M_2) \subseteq \Sigma^*$. Is $L(M_2) \subseteq L(M_1)$, the reverse, or neither? Justify it from the two transition tables. (Hint: look only at the last one or two symbols of w – the turnstile is more permissive than it looks.)

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (SIPSER) §1.1 “Finite Automata” – the formal definition, worked examples, and the formal definition of computation on a finite automaton, covering the same 5-tuple this document builds in Section [1.3](#), Sipser’s way.
2. (HMU) §2.1 “An Informal Picture of Finite Automata” – HMU’s own motivating, device-with-memory example before formalizing, the same role Section [1.1](#)’s turnstile plays here.
3. (HMU) §2.2 “Deterministic Finite Automata” – the formal 5-tuple definition together with both the transition-table and transition-diagram notations, matching Section [1.3](#) of this document.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

KVA SNUC

1 Constructing Deterministic Finite Automata

Lecture 5 asked one central question – what information about the past does a machine need to remember, in order to decide correctly about the future – and answered it once, with a turnstile and then with the formal deterministic finite automaton (DFA). This document turns that question into a working method: given a language L , described only in English, how do you actually build a DFA that recognizes it? We first place the DFA next to its more permissive sibling, the nondeterministic finite automaton (NFA), just to see the shape of what determinism buys you. Then the real content: a four-step recipe for building any DFA, applied first to the simplest possible languages (strings ending in a fixed symbol), then to four related languages built around a single pattern, **aba**, each construction reusing and adapting the last.

1.1 DFA and NFA: Two Members of One Family

Lecture 5's DFA had one non-negotiable rule: from every state, on every input symbol, there is exactly one next state. That single-valuedness is what makes δ a genuine function $Q \times \Sigma \rightarrow Q$.

Recap: DFA (Lecture 5)

$M = (Q, \Sigma, \delta, q_0, F)$ with $\delta : Q \times \Sigma \rightarrow Q$ – exactly one next state for every state-symbol pair.

Relax that one rule and you get an entire family of automata; this lecture only needs its loosest member named.

Nondeterministic finite automaton (NFA)

An **NFA** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$ – same shape as a DFA, except $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$: from a given state and symbol, δ names a *set* of next states, possibly empty, possibly more than one. N **accepts** w if *some* sequence of choices consistent with δ ends in a state in F .

The shape of the difference is all this lecture needs: a DFA's δ always returns one state, an NFA's δ returns a set. Turning an NFA into an equivalent DFA – and why nondeterminism never actually buys extra recognizing power – is a construction worth its own careful treatment later.

Exercises: Spotting Nondeterminism

1. For $\Sigma = \{a, b\}$ and $Q = \{q_0, q_1\}$: is $\delta(q_0, a) = \{q_0, q_1\}$ consistent with a DFA, an NFA, or neither? What about $\delta(q_1, b) = \emptyset$? (Hint: check each against the recap box and the NFA box above, one at a time.)
2. A friend claims every DFA is also an NFA, just written differently. Are they right? (Hint: compare what δ is allowed to return in each definition.)

1.2 A Method for Building a DFA

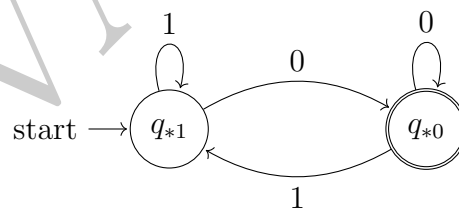
Definitions are cheap; the actual skill this lecture is after is turning a language description in English into a working DFA. That turns out to always be the same four moves.

Method: building a DFA from a language description

1. Decide what you must remember about the string read so far to know whether it is in L – Lecture 5’s central question, applied.
2. Turn every distinct answer to that question into a state, named after what it means.
3. For each state and each input symbol, work out what the answer becomes – that is δ .
4. F = the states whose meaning is “yes, so far”; q_0 = the state matching the empty string.

Apply it to $L_0 = \{w \in \{0,1\}^* : w \text{ ends with } 0\}$. Step 1: all you need to remember is the last symbol read, or that none has been read yet. Step 2: two states, named for exactly that meaning – q_{*0} (“last symbol read was 0”) and q_{*1} (“last symbol read was 1, or nothing yet”). Steps 3 and 4 are one table:

δ	0	1
q_{*0}	q_{*0}	q_{*1}
q_{*1}	q_{*0}	q_{*1}



with $q_0 = q_{*1}$ and $F = \{q_{*0}\}$. Notice δ above does not care what you plan to accept – it just tracks the last symbol read. $L_1 = \{w : w \text{ ends with } 1\}$ reuses the identical δ table and diagram; the only change is $F = \{q_{*1}\}$ and start state q_{*0} instead. The states already carried enough memory for both languages – accepting is just a choice of which memory values count as “yes.”

Exercises: One Method, New Languages

1. Using the same q_{*0}/q_{*1} states, what would F and q_0 need to be for $L = \{w : \text{the first symbol of } w \text{ is } 0\}$? Does the δ table above still work, or do you need new states? (Hint: “first symbol” needs memory of something this δ does not track.)
2. Apply the four-step method to $L = \{w \in \{0,1\}^* : |w| \text{ is even}\}$. How many states do you need, and what does each one mean?
3. Trace the ends-with-0 automaton above on $w = 0110$ and on $w = 0111$. Which is accepted?

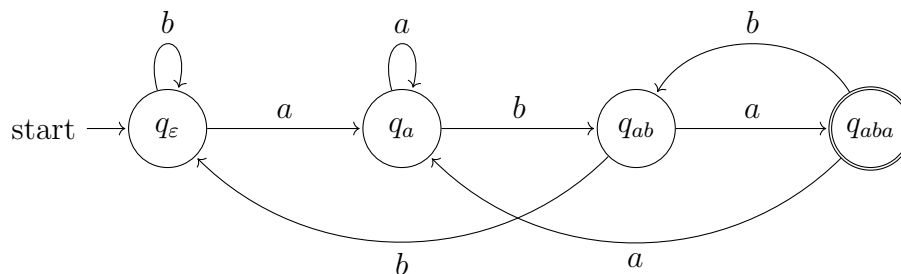
1.3 Four DFAs for One Pattern: aba

Now scale the method up to a real substring pattern over $\Sigma = \{a, b\}$. For all four languages below, Step 1’s answer is the same: remember how much of **aba** you are currently in the middle of matching – the longest suffix of the string read so far that is also a prefix of **aba**. That can only ever be ε , **a**, **ab**, or the whole **aba**, so four states cover it – and Step 2 says to name them after exactly that meaning: $q_\varepsilon, q_a, q_{ab}, q_{aba}$, for *matched so far: nothing, a, ab, aba*. Three transitions follow immediately from that meaning and never change across all four languages below: $\delta(q_\varepsilon, a) = q_a$, $\delta(q_a, b) = q_{ab}$, $\delta(q_{ab}, a) = q_{aba}$ – each one just advances the match by one symbol.

Ends with aba

The remaining entries handle what happens when a symbol *breaks* the match, and a broken match can still partially overlap the pattern: after **aba** itself, reading another **a** leaves “**abaa**”, whose longest suffix matching a prefix of **aba** is just **a** – back to q_a , not q_ε .

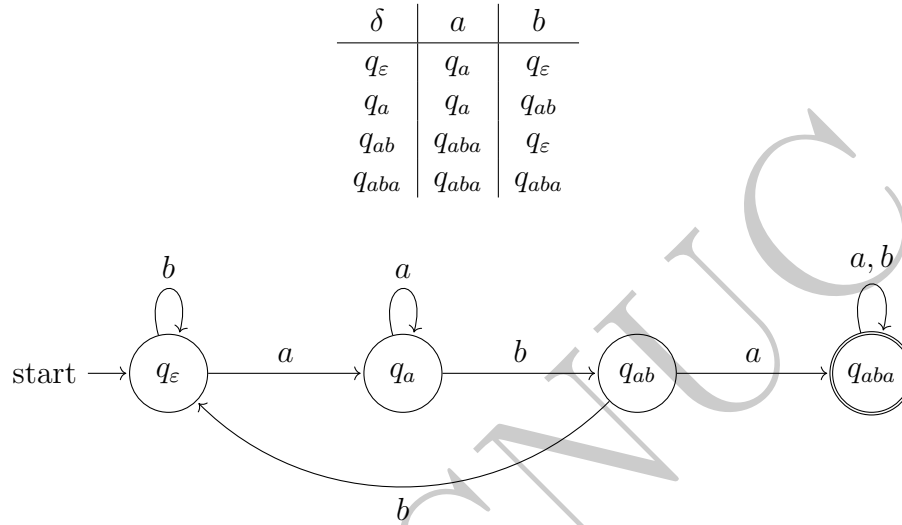
δ	a	b
q_ε	q_a	q_ε
q_a	q_a	q_{ab}
q_{ab}	q_{aba}	q_ε
q_{aba}	q_a	q_{ab}



Trace $w = \mathbf{abaaba}$: $q_\varepsilon \rightarrow q_a \rightarrow q_{ab} \rightarrow q_{aba} \rightarrow q_a \rightarrow q_{ab} \rightarrow q_{aba}$, ending in $F = \{q_{aba}\}$ – **accept**, and the fallback $\delta(q_{aba}, a) = q_a$ is exactly what made the second **aba** still get counted correctly.

Contains aba

Once **aba** has appeared *anywhere*, the string already qualifies, so there is nothing left to track – make q_{aba} absorbing: $\delta(q_{aba}, a) = q_{aba}$, $\delta(q_{aba}, b) = q_{aba}$ (was q_a, q_{ab} above). Every other entry is unchanged.

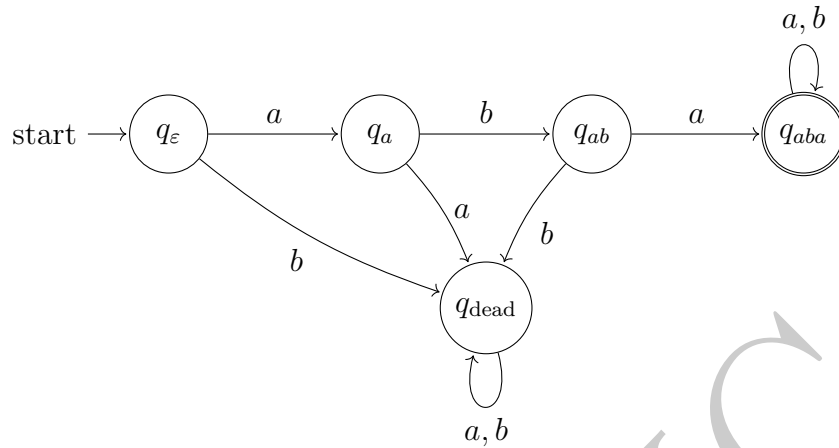


Trace $w = \mathbf{baabab}$: $q_\varepsilon \rightarrow q_\varepsilon \rightarrow q_a \rightarrow q_a \rightarrow q_{ab} \rightarrow q_{aba} \rightarrow q_{aba}$ – **aba** showed up in the middle (positions 3–5), q_{aba} was reached one symbol early, and the trailing **b** could not undo it. Ending in $F = \{q_{aba}\}$ – **accept**.

Starts with aba

Here a broken match is fatal, not partial: deviating from **a,b,a** at position 0, 1, or 2 means the string can *never* start with **aba**, so there is no overlap to fall back on – add a state named for exactly that outcome, q_{dead} , and send every failure there instead of back into $q_\varepsilon, q_a, q_{ab}$: $\delta(q_\varepsilon, b) = \delta(q_a, a) = \delta(q_{ab}, b) = q_{\text{dead}}$, with $\delta(q_{\text{dead}}, a) = \delta(q_{\text{dead}}, b) = q_{\text{dead}}$. Past the prefix, q_{aba} stays absorbing exactly as in the “contains” case, since anything may follow a confirmed **aba** prefix.

δ	a	b
q_ε	q_a	q_{dead}
q_a	q_{dead}	q_{ab}
q_{ab}	q_{aba}	q_{dead}
q_{aba}	q_{aba}	q_{aba}
q_{dead}	q_{dead}	q_{dead}

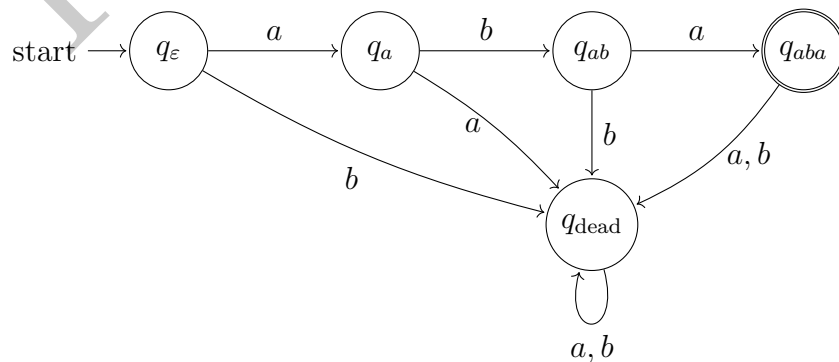


Trace $w = \mathbf{ababb}$: $q_\epsilon \rightarrow q_a \rightarrow q_{ab} \rightarrow q_{aba} \rightarrow q_{aba} \rightarrow q_{aba}$, ending in $F = \{q_{aba}\}$ – **accept** (the first three symbols are **aba**, and what follows does not matter). Trace $w = \mathbf{aab}$: $q_\epsilon \rightarrow q_a \rightarrow q_{\text{dead}} \rightarrow q_{\text{dead}}$ – **reject**: the second symbol broke the required prefix for good.

Exactly aba

Reuse the “starts with” automaton’s q_{dead} for the same reason – no partial credit for a broken prefix – but now q_{aba} must be fatal too, since nothing may follow: $\delta(q_{aba}, a) = q_{\text{dead}}$, $\delta(q_{aba}, b) = q_{\text{dead}}$ (instead of the absorbing self-loops above). Only three transitions in the whole table now lead anywhere but q_{dead} : $\delta(q_\epsilon, a) = q_a$, $\delta(q_a, b) = q_{ab}$, $\delta(q_{ab}, a) = q_{aba}$ – the single path spelling out **a,b,a** exactly, with every other move punished.

δ	a	b
q_ϵ	q_a	q_{dead}
q_a	q_{dead}	q_{ab}
q_{ab}	q_{aba}	q_{dead}
q_{aba}	q_{dead}	q_{dead}
q_{dead}	q_{dead}	q_{dead}



Trace $w = \mathbf{aba}$: $q_\epsilon \rightarrow q_a \rightarrow q_{ab} \rightarrow q_{aba}$, ending in $F = \{q_{aba}\}$ – **accept**. Trace $w = \mathbf{abab}$: $q_\epsilon \rightarrow q_a \rightarrow q_{ab} \rightarrow q_{aba} \rightarrow q_{\text{dead}}$ – **reject**: the fourth symbol is one symbol too many, and q_{aba} ’s only moves now lead straight to q_{dead} .

To think!!

All four automata above only ever mention a and b . Suppose Σ actually had a third symbol, c . Each automaton must still be a total function – some entry for c is needed at every state. For each of the four languages, what should $\delta(q, c)$ be at each state, and does answering that change any of the a/b entries already fixed above?

Exercises: Apply the Method

1. $L = \{w \in \{0,1\}^* : w \text{ has an even number of 0's and an even number of 1's}\}$. What must a state remember now? Compare the state count you need here against Section 1.2's two-state ends-with-0/ends-with-1 automata – why does tracking two independent conditions at once cost you a *product* of state counts, not a sum? (Hint: a state must record *both* parities simultaneously.)
2. $L = \{w \in \{0,1\}^* : w \text{ has at most three 1's}\}$. Build the DFA: how many states do you need to count 1's up to the point where one more would break the rule, and what does δ do once a fourth 1 has been read?
3. Over $\Sigma = \{0,1,\dots,9\}$, let $\text{atoi}(w)$ be the decimal number w spells out. Build a DFA for $L = \{w : \text{atoi}(w) \bmod 3 = 0\}$. (Hint: since $10 \equiv 1 \pmod{3}$, appending digit d to a number with remainder r gives a new remainder of $(r + d) \bmod 3$ – three states are enough.)

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (SIPSER) §1.2 “Nondeterminism” – the formal NFA definition, matching Section 1.1's NFA box, together with the equivalence of NFAs and DFAs this document only names and defers.
2. (HMU) §2.3 “Nondeterministic Finite Automata” – HMU's own NFA definition, the same role as the entry above.
3. (SIPSER) §1.1 “Finite Automata” – besides the 5-tuple already cited in Lecture 5's notes, this section's worked examples are exactly “designing finite automata,” the same skill Section 1.2's four-step method and Section 1.3's four **aba** automata build directly.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

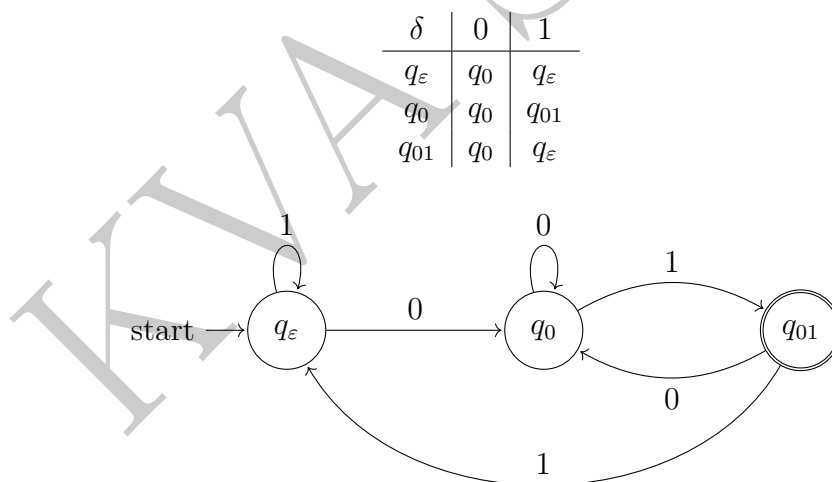
KVA SNUC

1 Nondeterministic Finite Automata

Lecture 6 named the nondeterministic finite automaton (NFA) just to place it next to the DFA and moved on – δ returns a set instead of a single state, full stop. Written out formally, an NFA is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$ – same shape as a DFA, except now $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$, the *power set* of Q : from a given state and symbol, δ no longer names one next state but a whole *set* of them, possibly empty, possibly more than one. This document is where that one-line difference earns its keep. We build the same ends-with-a-fixed-suffix language twice, once as a DFA and once as an NFA, so you can feel directly how much design effort nondeterminism buys back. Then we look inside an NFA’s computation as a tree of guesses, some of which run off the edge of δ and simply die. Armed with that picture we build a small gallery of NFAs – including a couple of genuinely tricky ones – and finally push δ one step further, letting a machine move between states for free, without reading any symbol at all: the ε -transition, and the ε -NFA it gives rise to.

1.1 Ends with 01: A DFA, Then an NFA

Take $L = \{w \in \{0,1\}^* : w \text{ ends with } 01\}$ over $\Sigma = \{0,1\}$. Lecture 6’s four-step method still applies – what must a state remember? Exactly the longest suffix of the string read so far that is also a prefix of 01: that can only be ε , 0, or the whole 01, giving three states $q_\varepsilon, q_0, q_{01}$.

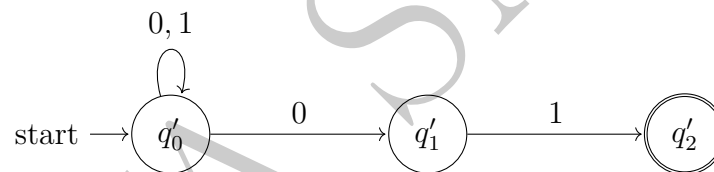


Every entry in that table earns its place by tracking memory that could, in principle, matter later in the string – that is exactly what a DFA is forced to do, since it commits to one next state and must be prepared for whatever comes next. An NFA is allowed to cheat: instead of tracking every way the suffix-so-far might still turn into 01, it can simply *wait*, and the instant it sees a 0 that might be the second-to-last symbol, *guess* that it is, and check.

Method: building an NFA by guessing

1. Find the one fact a DFA would have to remember exactly. Instead, design a state that *guesses* the fact the moment the evidence for it appears.
2. One state per stage of the guess: *haven't guessed yet*, *guessed*, *stage 1 of the pattern confirmed*, ..., *guess fully confirmed*.
3. The *haven't guessed yet* state keeps a self-loop on every symbol – it might always be wrong – and *additionally* branches into *stage 1* whenever the current symbol matches what a correct guess would need.
4. If the guess turns out wrong partway through, just leave that transition **undefined**. You never have to redirect a failed guess anywhere; the branch simply dies.

Applied to L : q'_0 (“haven’t guessed the suffix has started”) self-loops on both symbols, and additionally guesses on a 0 that this might be the second-to-last symbol, moving to q'_1 . From q'_1 , only a 1 confirms the guess, landing in the accepting q'_2 ; a 0 from q'_1 means the guess was wrong, and $\delta(q'_1, 0)$ is simply left undefined.



Three states either way, but notice what disappeared: no entry ever had to say “and if instead we see this, go back to remembering nothing.” The NFA only ever describes forward progress or silent death – never a controlled retreat, which is exactly the bookkeeping a DFA cannot avoid.

A note on where nondeterminism comes from

Nondeterministic automata were introduced by Michael Rabin and Dana Scott in their 1959 paper *Finite Automata and Their Decision Problems* – the same paper that first proved an NFA never recognizes more languages than a DFA can, a result this document only uses and a later one proves. The paper was part of the body of work that earned both of them the 1976 Turing Award.

Exercises: Ends-With Automata

1. Build the NFA for $L = \{w \in \{0,1\}^* : w \text{ ends with } 10\}$ by the same guessing method. How many states, and which transition is left undefined? (Hint: mirror every 0 and 1 in the q'_0, q'_1, q'_2 machine above.)
2. Trace the NFA above on $w = 110$ and on $w = 1011$ by writing down, after each symbol, the *set* of states the machine could be in. Which strings are accepted?
3. A classmate claims the NFA above still needs a transition $\delta(q'_1, 0) = q'_0$, “just in case.” Explain, using δ ’s definition from Lecture 6’s NFA box, why leaving it undefined is not an error.

1.2 A Small Computation Tree: Where Branches Die

An NFA does not run one computation on an input; it runs *all* of them at once, one branch per choice available at each step. Naming that formally just means extending δ from one symbol to a whole string:

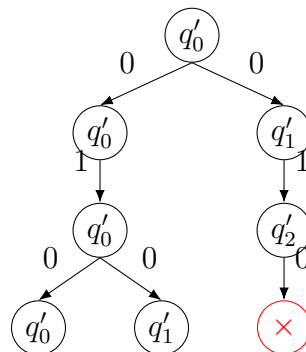
The extended transition function $\hat{\delta}$

$\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$ is built up one symbol at a time:

$$\hat{\delta}(q, \varepsilon) = \{q\}, \quad \hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q, x)} \delta(p, a) \quad \text{for } x \in \Sigma^*, a \in \Sigma.$$

N accepts w iff $\hat{\delta}(q_0, w) \cap F \neq \emptyset$.

$\hat{\delta}(q_0, w)$ is exactly the set of leaves the computation tree below reaches at depth $|w|$ – every branch that ran off the edge of δ before reaching depth $|w|$ is simply absent from that set, contributing nothing to whether the intersection with F is empty. Trace the q'_0, q'_1, q'_2 machine from Section 1.1 on $w = 010$, drawing every choice as a branch of a tree rooted at q'_0 :



Read the tree level by level: after the first 0, the machine genuinely forks, since $\delta(q'_0, 0) = \{q'_0, q'_1\}$ – one branch keeps not-guessing, the other commits. After the second symbol, 1, the right branch’s guess looks confirmed ($q'_1 \rightarrow q'_2$), but the third symbol is 0, and $\delta(q'_2, 0)$ was

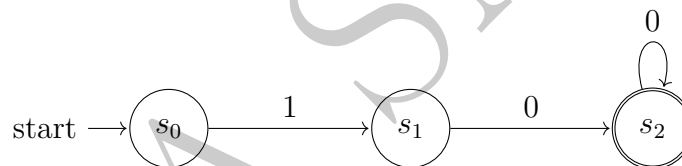
never defined – that whole branch dies, marked $\times$, and is simply dropped from consideration. It is *not* a rejection of the string by itself; it is one guess admitting it was wrong. The string is only rejected once *every* surviving leaf – here q'_0 and q'_1 – misses $F = \{q'_2\}$, which is exactly what happens: 010 ends in 10, not 01, so **reject**, and the tree agrees with the table you would have built by hand. In symbols: $\hat{\delta}(q'_0, 010) = \{q'_0, q'_1\}$, and since that set misses $F = \{q'_2\}$, N rejects 010 – the tree is simply $\hat{\delta}$'s recursion drawn out one branch at a time.

To think!!

Suppose, instead, every single branch of a computation tree died before reaching depth $|w|$ – not one of them survives to the last symbol. Is w accepted or rejected? Compare this against a branch that survives to depth $|w|$ but lands in a non-accepting state – are these two situations the same outcome for a different reason, or genuinely different?

1.3 A Concatenation Needs No Guessing

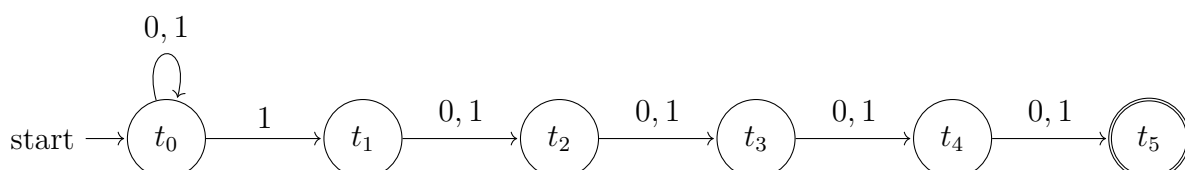
Not every NFA needs a dramatic guess. Take $L_1 = \{10^{n+1} : n \geq 0\}$ – a single 1 followed by one or more 0's. Step 1 of the guessing method finds nothing to guess at all: every symbol is forced, so δ is already a function in disguise.



Useful mainly as a reminder that “build an NFA” does not mean “look for something to guess” – sometimes the language just is a straight-line pattern with a star tacked on the end, and δ reflects that directly.

1.4 The Flagship Example: Guessing at a Distance

L_1 made nondeterminism look almost optional. The next language is where it stops being optional: $L_2 = \{w \in \{0,1\}^* : \text{the fifth symbol from the end of } w \text{ is } 1\}$. A DFA for L_2 must remember the last five symbols read, since it cannot know in advance which one will turn out to be fifth-from-last once the string ends – that forces $2^5 = 32$ states, one per possible 5-symbol window. An NFA does not need to remember the window at all; it just *guesses*, on some 1, that this is the fifth-from-last symbol, and then insists on exactly four more symbols of any kind before the string ends.



Six states, none of them exponential in anything: t_0 is “still waiting to guess,” and every guess that turns out four symbols too early or too late is simply a branch that never lands in t_5 – it does not need a name, a fallback, or a transition back to t_0 . That gap, thirty-two states against six, *is* the case for nondeterminism.

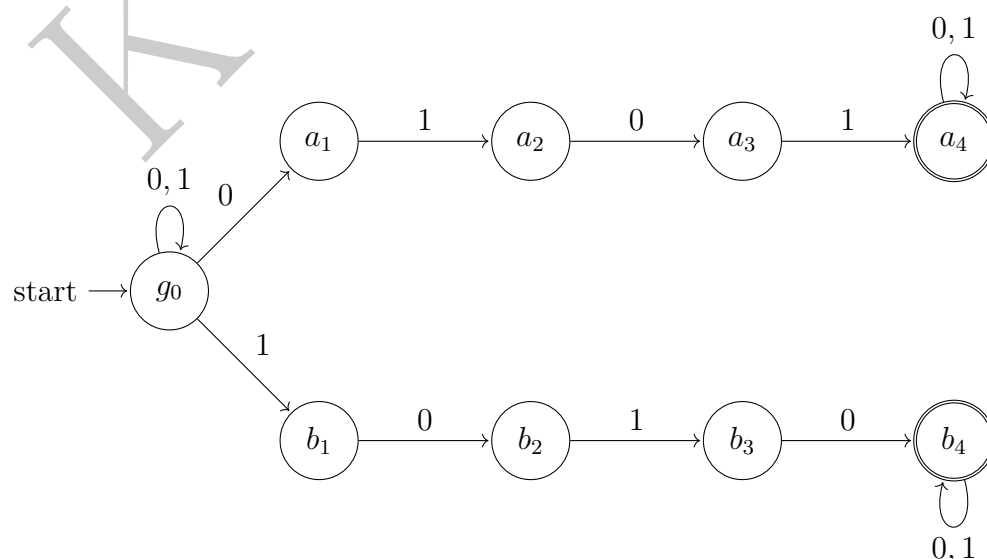
Exercises: Guessing at a Distance

1. Build the NFA for “the third symbol from the end is 0” by the same recipe as L_2 . How many states, and how does the count scale as “fifth” becomes “ k -th”? (Hint: count the forced any-symbol stages after the guess.)
2. Trace the L_2 automaton above on $w = 11000$ and on $w = 10000$. Which is accepted, and which single guessed transition made the difference?
3. A DFA for “the third symbol from the end is 0” needs $2^3 = 8$ states by the sliding-window argument above. Confirm this by naming what each of the 8 states must remember.

1.5 A Gallery of Harder NFAs

The examples so far each need one guess. Real languages often combine several conditions, and an NFA handles *or* beautifully – just branch into a separate small machine for each alternative and let them run in parallel; the string is accepted the moment any one of them succeeds.

G1: contains 0101 or contains 1010, over $\{0, 1\}$. Keep a start state g_0 self-looping on both symbols (“haven’t started matching either pattern”), and hang two independent four-stage chains off it, one per pattern, each ending in an absorbing accept state – absorbing exactly as in Lecture 6’s “contains aba” automaton, since one occurrence anywhere is enough.



The b_1, b_2, b_3, b_4 chain off g_0 on a first 1 matches 1010 the same way as the a -chain matches 0101, just with every 0 and 1 swapped. A wrong symbol anywhere mid-chain (say a_2 reading a 1 instead of the required 0) is, once again, just an undefined transition – the branch dies and the other chain is unaffected.

G2: exactly two 1's, or length exactly 4, over $\{0, 1\}$. These two conditions share nothing, so branch at the very start into two independent counters. g_0 doubles as “0 ones read” for the first condition (self-loop on 0), and additionally spawns a length-counter on *every* symbol.

δ	0	1
g_0	$\{g_0, b_1\}$	$\{a_1, b_1\}$
a_1	a_1	a_2
a_2 (accept)	a_2	$\emptyset$
b_1	b_2	b_2
b_2	b_3	b_3
b_3	b_4	b_4
b_4 (accept)	$\emptyset$	$\emptyset$

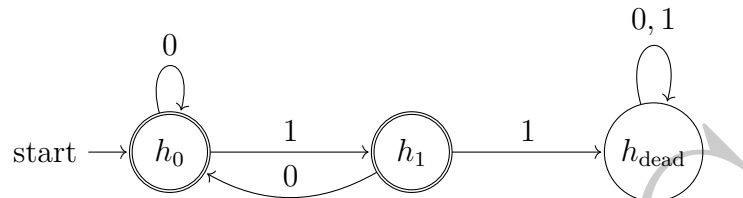
a_2 's empty entry is the “exactly two, not three or more” guardrail: a third 1 kills that branch, exactly as q_{dead} -flavoured entries did in Lecture 6, except here there is no explicit dead state to name – the branch just stops existing. A DFA for the same language would need to track *both* the running 1-count and the running length in a single state, a product of the two – nondeterminism avoids the product entirely by never merging the two conditions into one state in the first place.

G3: third symbol from the start is 1, or third symbol from the end is 1, over $\{0, 1\}$. The first half is a plain deterministic count; the second half is exactly Section 1.4's guessing trick with two forced stages instead of four. g_0 runs *both* simultaneously: on every symbol it keeps self-looping (still waiting to guess the end-condition) and advances a start-counting chain $c_1 \rightarrow c_2$, while also branching into the end-guess chain e_1 whenever the current symbol is 1.

δ	0	1
g_0	$\{g_0, c_1\}$	$\{g_0, c_1, e_1\}$
c_1	c_2	c_2
c_2	$\emptyset$	c_3 (accept, absorbing)
e_1	e_2	e_2
e_2	e_3 (accept, absorbing)	e_3 (accept, absorbing)

Notice g_0 alone spawns up to three simultaneous branches on a single 1 – itself, the start-counter, and a fresh end-guess – which is precisely the kind of bookkeeping that would force a DFA into a large product construction.

G4: does *not* contain 11, over $\{0, 1\}$ – a trap. It is tempting to expect a clever guess here too, since “contains” languages love nondeterminism. But *avoiding* a pattern needs no guessing at all – every branch must avoid it, so there is only ever one sensible next state, and the “NFA” below is a DFA wearing an NFA’s clothes.

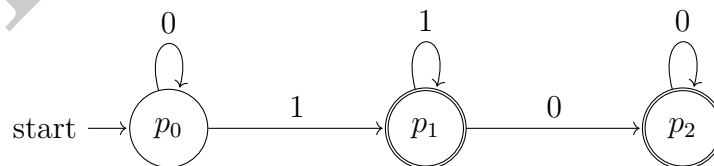


This is worth stating plainly, not just leaving to the exercise below: unlike a DFA, an NFA cannot be complemented just by swapping which states are accepting. A DFA’s $\hat{\delta}$ tracks one unique state per string, so flipping F flips the answer for every string at once. An NFA’s $\hat{\delta}$ tracks a whole *set* of branches, some of which die outright – and Section 1.2 already established that a dead branch is not a rejecting one, it is simply gone, so swapping F has no well-defined effect on it.

To think!!

Suppose you tried to build “does not contain 11” by taking the *contains-11* NFA (self-looping start state u_0 , a guess branch $u_0 \rightarrow u_1$ on a 1, and an absorbing accept state u_2 once u_1 sees a second 1) and just swapping which states are in F . Trace it on $w = 110$: the guess branch u_1 dies on the trailing 0, but u_0 and the already-confirmed u_2 both survive to the end, so the frontier is $\{u_0, u_2\}$. Does swapping F from $\{u_2\}$ to $\{u_0, u_1\}$ compute the right answer for whether $w = 110$ avoids 11? (Hint: u_0 surviving alongside u_2 is not a coincidence – u_0 never had a reason to die, guess or no guess. What does that mean for any string that already contains 11 but keeps going?)

G5: exactly one maximal block of 1’s, over $\{0, 1\}$ – i.e. $w = 0^i 1^j 0^k$ with $j \geq 1$. Three pieces chained in sequence, forced throughout, no branching needed:



p_0 is deliberately *not* accepting – $j \geq 1$ rules out the all-zero string. Keep this picture in mind; it is exactly the shape Section 1.6 below will build by gluing three tiny independent pieces together instead of hand-designing one three-state chain.

Exercises: Build Your Own

1. Build an NFA for $\{w \in \{0,1\}^* : w \text{ contains } 00 \text{ or } w \text{ contains } 11\}$ – how much of G1’s construction can you reuse directly? (Hint: two two-stage chains off one shared start state.)
2. Build an NFA for $\{w \in \{a,b\}^* : \text{2nd symbol is } a \text{ and } w \text{ ends with } b\}$. Is this an *or* of independent conditions like G2/G3, or does it need a genuine *and*? Does that change how naturally nondeterminism helps?
3. Compare your answer to Exercise 1 against G1: does “contains 00 or 11” need the absorbing-accept-state trick the same way “contains 0101 or 1010” did? Why or why not?

1.6 Epsilon-Closures and ε -Transitions

Every NFA above still only moves when it reads a symbol. Relax that too, and let δ include a transition labelled ε – one the machine may take for free, without consuming any input at all. The immediate question this raises: if a machine can drift through any number of free moves before or after reading a real symbol, what does it even mean to say “the machine is in state q ”? The honest answer is that it is in *every* state reachable from q by zero or more ε -moves, all at once.

Epsilon-closure

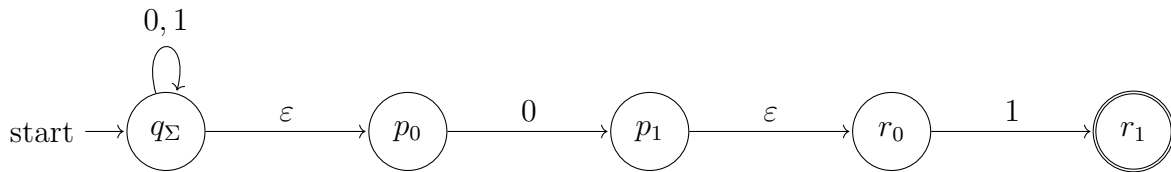
For a state q , $E(q)$ is the set of all states reachable from q using zero or more ε -transitions – in particular $q \in E(q)$ always, since taking “zero” free moves is allowed. For a set of states S , $E(S) = \bigcup_{q \in S} E(q)$. (Sipser calls this function E ; HMU calls the same object **ECLOSE** – same idea, different textbook, different letter.)

Reading a real symbol in an ε -NFA is the same two-step dance every time: first take the ordinary δ -move out of every state currently in play, then let the result drift through as many free moves as it likes before calling the frontier settled. Written out, that dance is:

ε -NFA

An ε -**NFA** is $N = (Q, \Sigma, \delta, q_0, F)$ with $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$. Reading a string $w = a_1 a_2 \cdots a_n$ starts at the frontier $E(\{q_0\})$ – not just $\{q_0\}$ – and after each symbol a_i the new frontier is $E\left(\bigcup_{q \in \text{old frontier}} \delta(q, a_i)\right)$. N accepts w if the final frontier intersects F .

E is worth computing on a real machine rather than a toy one, so build the ends-with-01 language a third way, this time by gluing three tiny, independently obvious pieces with ε -edges instead of hand-designing one clever three-state NFA: a fragment for “any string at all,” q_Σ , self-looping on 0 and 1; glued by ε to a fragment for the literal symbol 0, $p_0 \rightarrow p_1$; glued by ε to a fragment for the literal symbol 1, $r_0 \rightarrow r_1$, accepting.



$E(q_\Sigma) = \{q_\Sigma, p_0\}$: one free hop along the ε -edge, and p_0 itself has no ε -edge out, so the closure stops there. Run $w = 0101$ through this machine, tracking the full frontier after each symbol:

after reading	frontier (already E -closed)
(nothing yet)	$\{q_\Sigma, p_0\}$
0	$\{q_\Sigma, p_0, p_1, r_0\}$
01	$\{q_\Sigma, p_0, r_1\}$
010	$\{q_\Sigma, p_0, p_1, r_0\}$
0101	$\{q_\Sigma, p_0, r_1\}$

The final frontier contains $r_1 \in F$ – **accept**. That matches both the ends-with-01 DFA of Section 1.1 and its raw NFA on the very same string $w = 0101$: all three machines, built three different ways, agree. What the ε -edges bought here is not a shorter machine – it is the same three states as the raw NFA – but an easier *design process*: nobody had to notice that q'_0 needed to double as both “matching nothing” and “still open to starting a match.” You design “any string,” “the symbol 0,” and “the symbol 1” separately, as three obviously-correct fragments, and wire them together after the fact.

The same gluing trick makes L_1 from Section 1.3 almost too easy: a fragment for 1, ε -glued to a fragment for 0, ε -glued to a fragment for 0^* (one state, self-looping, accepting) – three independently trivial pieces, zero cleverness. The fifth-symbol-from-the-end machine of Section 1.4 gains less from gluing, since its four forced any-symbol stages were never in need of a guess to begin with; the only place ε helps there is snapping the generic “any prefix” fragment q_Σ onto the front of the guess-and-count chain with a single wire, the same role it played above.

Exercises: Closures and Gluing

1. For the ends-with-01 ε -NFA above, compute $E(p_1)$ and $E(r_0)$ by hand. (Hint: check every state each one reaches by *zero or more* ε -edges, not just one hop.)
2. Build G5 from Section 1.5 ($0^i 1^j 0^k$, $j \geq 1$) as three ε -glued fragments – one for 0^* , one for 1^+ (one or more 1’s), one for 0^* again. Compare the total state count against the hand-designed three-state p_0, p_1, p_2 chain – did gluing cost you extra states this time, and if so, why might that be an acceptable trade?
3. Run your ε -NFA answer to Exercise 2 on $w = 0011000$ by tracking the E -closed frontier after every symbol, the same way the worked example above tracked $w = 0101$. Does it accept?

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (SIPSER) §1.2 “Nondeterminism” – the formal NFA definition Lecture 6 deferred, Sipser’s own guess-driven constructions for the regular operations (the same technique Sections 1.1–1.5 build up by hand), and the definition of $E(R)$ this document’s E notation follows.
2. (HMU) §2.3 “Nondeterministic Finite Automata” – HMU’s parallel treatment of raw NFAs, including the formal definition of acceptance via a computation tree used in Section 1.2.
3. (HMU) §2.5 “Finite Automata With Epsilon-Transitions,” specifically §2.5.3 “Epsilon-Closures” – the same object as Section 1.6’s E , under HMU’s own name for it, $ECLOSE$, and §2.5.4 on extending δ to strings through it, matching the frontier computation used on $w = 0101$ there.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 Equivalence of Finite Automata

Lecture 7 ended by building the same language three different ways – DFA, NFA, ε -NFA – and watching all three agree on $w = 0101$. That agreement was not a coincidence to admire and move past; it is the entire content of this lecture, made precise and then made mechanical. We first collapse an ε -NFA back down to a plain NFA, folding every free move directly into δ . Then we state – and argue straight from the definitions involved, no construction needed – why a DFA and an NFA can never disagree on what they recognize. The interesting half of that argument, turning an NFA into an equivalent DFA, gets its own construction: the subset construction, run on two machines from class. We close by asking a question the subset construction cannot answer by itself: given two DFAs that look nothing alike, how do you tell whether they recognize the same language – and, pushed one step further, how would you even know if a single DFA is carrying states it does not need at all?

1.1 From ε -NFA to NFA: Eliminating the Free Moves

An ε -NFA is convenient to *design* – Lecture 7 built one by gluing three obvious fragments together – but every definition of acceptance we actually compute with (Lecture 7’s computation tree, the subset construction later in this document) is stated for machines that only move on real symbols. So before doing anything else with an ε -NFA, it is worth asking: can we always trade it for a plain NFA that accepts exactly the same language? The answer is yes, and E – already defined in Lecture 7 – is exactly the tool that does it: fold every free move a state could take *before* and *after* reading a symbol directly into a new δ .

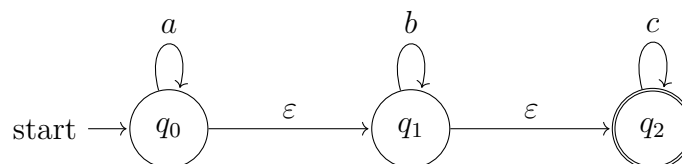
Eliminating ε -transitions

Given an ε -NFA $N = (Q, \Sigma, \delta, q_0, F)$, build a plain NFA $N' = (Q, \Sigma, \delta', q_0, F')$ with the *same* states, start state, and language:

$$\delta'(q, a) = E\left(\bigcup_{p \in E(q)} \delta(p, a)\right) \quad \text{for every } q \in Q, a \in \Sigma,$$

and $F' = F \cup \{q_0\}$ if $E(q_0) \cap F \neq \emptyset$, and $F' = F$ otherwise – the one adjustment needed to correctly handle the case where N would have accepted ε itself for free.

Try it on the ε -NFA for $L = \{a^i b^j c^k : i, j, k \geq 0\}$ – three self-looping states glued in a line, one letter each, exactly the shape Lecture 7 closed with:



E is a straight walk forward along the chain of free moves, so it costs nothing to write down for every state at once:

$$E(q_0) = \{q_0, q_1, q_2\}, \quad E(q_1) = \{q_1, q_2\}, \quad E(q_2) = \{q_2\}.$$

Feed those into the boxed formula, one (q, x) pair at a time, to get δ' :

δ'	a	b	c
q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
q_1	$\emptyset$	$\{q_1, q_2\}$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$	$\{q_2\}$

One adjustment is still owed: $E(q_0) = \{q_0, q_1, q_2\}$ already contains the accepting q_2 , meaning the original ε -NFA accepts ε itself (sensible – $a^0b^0c^0$ is a legal string of the language) – so q_0 joins F in the new machine: $F' = \{q_0, q_2\}$. Trace $w = \mathbf{ab}$ through N' as a check: $\{q_0\} \xrightarrow{a} \{q_0, q_1, q_2\} \xrightarrow{b} \{q_1, q_2\}$, and $q_2 \in F'$ – **accept**, correctly. Trace $w = \mathbf{ba}$: $\{q_0\} \xrightarrow{b} \{q_1, q_2\} \xrightarrow{a} \emptyset$ – **reject**, since neither q_1 nor q_2 has an a -move, exactly as it should be for a string that puts a **b** before the **a**.

Exercises: Eliminating ε

1. Apply the elimination algorithm to Lecture 7's ends-with-01 ε -NFA (states $q_\Sigma, p_0, p_1, r_0, r_1$). Do you recover exactly Lecture 7's hand-built raw NFA q'_0, q'_1, q'_2 , or a machine that merely agrees with it on every string? (Hint: compare state *counts* first, then compare δ entry by entry.)
2. In the $a^*b^*c^*$ example above, $E(q_1)$ also contains the accepting q_2 – yet q_1 was *not* added to F' . Explain why that omission is correct. (Hint: the F' -fix in the boxed algorithm is stated only for q_0 – what is special about the start state that makes the fix unnecessary everywhere else?)
3. Trace $w = \mathbf{aabbcc}$ and $w = \mathbf{ac}$ through the N' built above. Are both accepted? Should they be, given $L = \{a^ib^jc^k\}$?

1.2 DFA and NFA Recognize the Same Languages

Every DFA in this course so far has secretly also been an NFA – nobody ever checked, because nobody needed to. Once δ 's codomain is officially $\mathcal{P}(Q)$ rather than Q , the two definitions sit close enough together that half of their equivalence needs no construction at all, only a careful re-reading of what each 5-tuple already requires.

Theorem: DFAs and NFAs are equally powerful

A language L is recognized by some DFA if and only if L is recognized by some NFA.

One direction is immediate from the definitions alone. Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA, so $\delta : Q \times \Sigma \rightarrow Q$ is a genuine function. Define $N = (Q, \Sigma, \delta', q_0, F)$ with $\delta'(q, a) = \{\delta(q, a)\}$ – the singleton containing M 's unique next state. Check N against Lecture 6's NFA box, clause by clause: same Q , same Σ , same q_0 , same F , and $\delta' : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ genuinely returns a subset of Q for every state-symbol pair – a subset that just happens to always have size exactly 1. Nothing in the NFA definition forbids that. Since N never has more than one state to choose from at any step, its one and only computation on any w is identical to M 's,

so $L(N) = L(M)$ – **every DFA-recognizable language is NFA-recognizable**, and the construction above needed nothing but the two definitions side by side.

The converse – every NFA-recognizable language is also DFA-recognizable – is the direction that actually does work, since an NFA’s δ is allowed to return 0, 2, or more states at once, and no DFA’s δ can mimic that directly. That is exactly what Section 1.3 builds.

Exercises: Reading the Definitions Closely

1. Which single clause of the NFA definition would $\delta'(q, a) = \{\delta(q, a)\}$ fail if M ’s δ were allowed to be a *partial* function (undefined on some state-symbol pairs), the way HMU sometimes allows for DFAs? (Hint: re-read $\mathcal{P}(Q)$ ’s definition – does it contain $\emptyset$?)
2. A classmate defines $\delta''(q, a) = \{\delta(q, a), q_0\}$ instead of the singleton above, calling it “a more interesting way to embed a DFA into an NFA.” Is $L(N'')$ (using δ'') still equal to $L(M)$ in general? (Hint: try it on the ends-with-0 DFA from Lecture 6 and a string that is not in $L(M)$.)

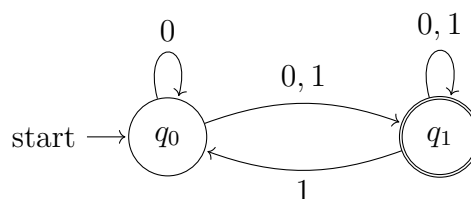
1.3 From NFA to DFA: The Subset Construction

An NFA’s δ can leave a DFA’s single-next-state promise behind entirely – at any step it might offer no next state, one, or several. The fix is almost embarrassingly direct: if an NFA can be in several states *at once*, just make each of those *simultaneous possibilities* into a single new DFA state. A DFA state is no longer one of the NFA’s states; it is a whole *set* of them.

The subset construction

Given an NFA $N = (Q, \Sigma, \delta, q_0, F)$, build a DFA $M = (Q_D, \Sigma, \delta_D, q_{0,D}, F_D)$ with $Q_D = \mathcal{P}(Q)$, $\delta_D(S, a) = \bigcup_{q \in S} \delta(q, a)$ for $S \subseteq Q$, $q_{0,D} = \{q_0\}$, and $F_D = \{S \subseteq Q : S \cap F \neq \emptyset\}$. Then $L(M) = L(N)$. In practice, build only the subsets *reachable* from $q_{0,D}$ by following δ_D – most of $\mathcal{P}(Q)$ is usually never visited.

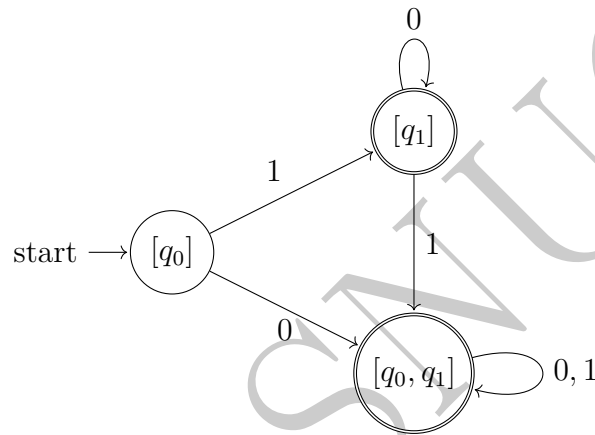
Worked Example A. Take the 2-state NFA below, q_0 the start state and q_1 the only accepting state.



With only 2 states, $\mathcal{P}(Q)$ has a mere 4 subsets, and building all of them costs nothing:

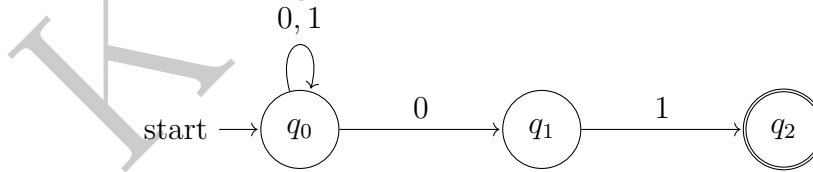
δ_D	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$[q_0]$ (start)	$[q_0, q_1]$	$[q_1]$
$[q_1]$ (accept)	$[q_1]$	$[q_0, q_1]$
$[q_0, q_1]$ (accept)	$[q_0, q_1]$	$[q_0, q_1]$

Every subset containing q_1 is accepting, exactly as F_D 's definition demands. $\emptyset$ never gets reached from $[q_0]$ here, so it is dropped from the diagram – the DFA that actually does the work has only 3 states:



Once the machine leaves $[q_0]$ it can never come back – every edge out of $[q_1]$ and $[q_0, q_1]$ lands in an accepting state – so this NFA (and its DFA) accepts every nonempty string over $\{0, 1\}$ and rejects only ε .

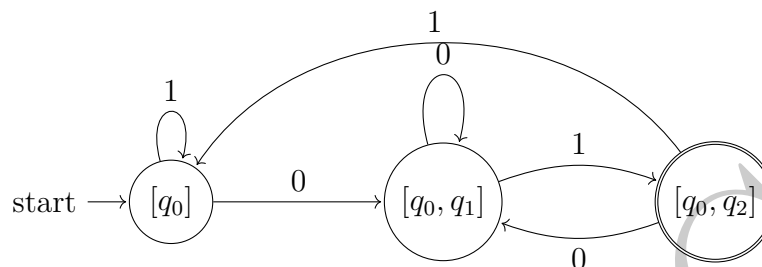
Worked Example B. A 3-state NFA makes the savings from “only build what’s reachable” concrete. q_0 is the start state, q_2 the only accepting one.



$\mathcal{P}(Q)$ now has 8 subsets. Building the full table is still cheap on paper, but watch which rows a walk starting from $\{q_0\}$ actually visits (checked $\checkmark$):

δ_D	0	1	reachable?
$\emptyset$	$\emptyset$	$\emptyset$	
$\{q_0\}$ (start)	$\{q_0, q_1\}$	$\{q_0\}$	$\checkmark$
$\{q_1\}$	$\emptyset$	$\{q_2\}$	
$\{q_2\}$	$\emptyset$	$\emptyset$	
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\checkmark$
$\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$	
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$	$\checkmark$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$	

Five of the eight rows above are never reached from $\{q_0\} - \{q_1\}$, $\{q_2\}$, $\{q_1, q_2\}$, and $\{q_0, q_1, q_2\}$ describe perfectly consistent subsets of Q , but no string ever drives the machine into them, so they earn no place in the DFA:

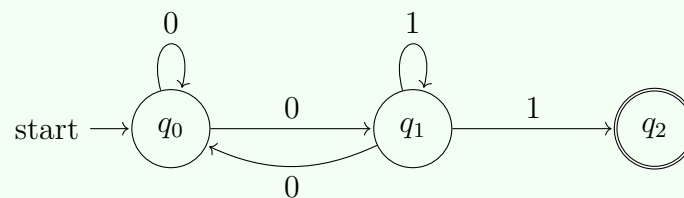


A note on how bad it can get

Only building reachable subsets does not mean the blow-up is a myth. HMU's own textbook treatment of the subset construction includes a section titled "A Bad Case for the Subset Construction," and the language it uses to force all 2^n subsets to be reachable is exactly Lecture 7's flagship family – "the k -th symbol from the end is 1." The same guess that made the NFA small in Lecture 7 is what forces the DFA to remember every one of the 2^k possible last- k windows – there is no way around it for *that* language, reachable-only or not.

Exercises: Building the Subsets

1. Convert the NFA below to a DFA, building only the reachable subsets.



2. In Worked Example A, $\emptyset$ appears in the full subset table but never in the final diagram. Under what circumstance *would* $\emptyset$ become reachable from $q_{0,D}$ for some NFA – what would it mean, in terms of Lecture 7's NFA computation tree, for the DFA to be sitting in state $\emptyset$?
3. Worked Example B's NFA and Worked Example A's NFA both reject ε . Find the shortest string accepted by exactly one of the two resulting DFAs – you will need this string again in Section 1.4's exercises.

1.4 Checking Whether Two DFAs Agree

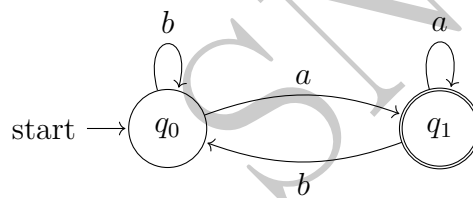
The subset construction settles *existence* – every NFA has some equivalent DFA – but says nothing about *comparing* two DFAs that were built independently and look nothing alike.

Given M_1 and M_2 , is $L(M_1) = L(M_2)$? Checking every string is not an option, but checking every *pair of states* is.

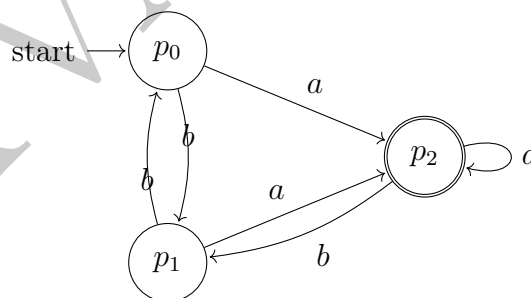
Method: the state-pair tree

1. Root the tree at the pair $(q_{0,1}, q_{0,2})$ – the two start states.
2. From a pair (p, q) , add one child $(\delta_1(p, a), \delta_2(q, a))$ per symbol $a \in \Sigma$.
3. The instant a pair (p, q) appears with exactly one of p, q accepting, stop: M_1 and M_2 **disagree**, and the input spelled out by the path from the root to (p, q) is a string one machine accepts and the other rejects.
4. If no such pair ever appears – every pair reachable this way agrees on accept/reject – then M_1 and M_2 are **equivalent**: $L(M_1) = L(M_2)$.

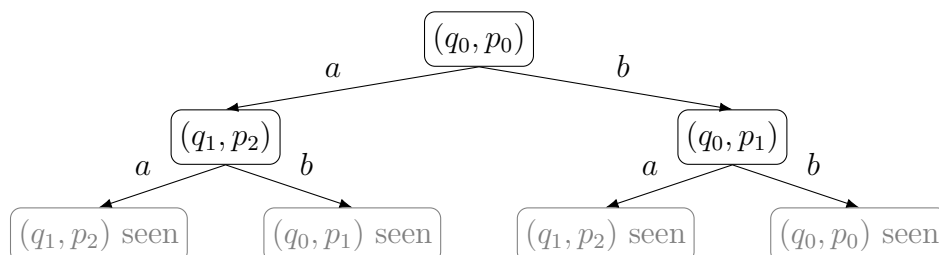
Try it on two DFAs over $\{a, b\}$ that were built with no thought of one another. M tracks only the last symbol read:



P recognizes the same idea – “ends in **a**” – but was built by someone who never simplified their own case split, keeping two separate non-accepting states instead of one:



Root the tree at (q_0, p_0) and expand:



Every pair that appears – (q_0, p_0) , (q_0, p_1) , (q_1, p_2) – has both coordinates accepting or both rejecting, and by depth 2 every child is a *repeat* of a pair already on the tree, so there is nothing new left to check: M and P **agree**, $L(M) = L(P)$, despite P carrying a state M never needed. Trace $w = \text{ababa}$ through both as a sanity check – both land in an accepting state, exactly as the tree predicts for any string ending in **a**. That extra state, p_1 , is not doing anything p_0 was not already doing: from either one, reading a lands in the same place (p_2), and reading b lands in the other one of the pair – p_0 and p_1 are, for every purpose that matters, the *same* state wearing two names.

Exercises: Comparing Machines

1. Run the state-pair tree method on Worked Example A's DFA and Worked Example B's DFA from Section 1.3, starting from the pair $([q_0]_A, [q_0]_B)$. How many levels deep does it take to find a disagreeing pair, and does the string you get match your answer to that section's Exercise 3?
2. Suppose two DFAs have a different number of states but the pair-tree method never finds a disagreeing pair. Are they still equivalent as *languages*? Are they equivalent as *machines* – is there any meaningful difference between the two questions?

To think!!

The state-pair tree above compares states *across* two different machines. Nothing stops you from running the very same method on a *single* DFA against itself – comparing pairs of its own states (q, q') with $q \neq q'$, to ask whether the machine, sitting in q , would from now on behave identically to the machine sitting in q' , no matter what string comes next. If two states never disagree on any suffix, what would stop you from merging them into one state and still recognizing the exact same language? And if that is true, what does it say about whether the DFAs you have been hand-building all course – states named for exactly what you decided to remember – were ever using the *fewest* states possible for their language, or just *a* correct number?

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (SIPSER) §1.2 “Nondeterminism” – Sipser's own equivalence proof (subset construction) and ε -NFA elimination, covering the same ground as Sections 1.1–1.3 in one

continuous argument.

2. (HMU) §2.3.5 “Equivalence of Deterministic and Nondeterministic Finite Automata” – the formal subset-construction theorem matching Section 1.3, including the proof by induction on $|w|$ this document only sketches informally.
3. (HMU) §2.5.5 “Eliminating ε -Transitions” – the elimination algorithm matching Section 1.1, written with HMU’s own name for E , ECLOSE, rather than Sipser’s E .
4. (HMU) §4.4.1 “Testing Equivalence of States” and §4.4.3 “Minimization of DFA’s” (both within §4.4 “Equivalence and Minimization of Automata”) – the formal treatment of Section 1.4’s state-pair method and the minimization question this document closes on without answering.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 DFA Minimization and the Myhill-Nerode Theorem

Lecture 8 closed by admitting a DFA might be carrying states it does not need, and left the observation dangling – is there always a smallest correct machine, and how would you actually find it? This lecture answers both halves. First, a precise notion of when two states of a DFA are secretly the same state wearing two names – *distinguishable* states – and an algorithm, the table-filling method, that finds every such pair mechanically. Then the theorem underneath all of it, Myhill-Nerode, which says the same idea one level up: two *strings* are equivalent exactly when no future can tell them apart, and it is this string-level equivalence, not the particular DFA you happened to draw, that fixes the true minimum number of states a language can ever be recognized with.

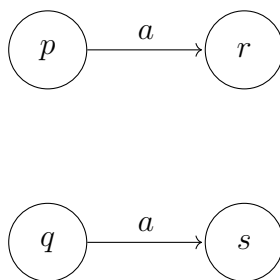
1.1 When Are Two States the Same?

Every DFA in this course so far was built by the four-step method of Lecture 6: decide what to remember, make a state per answer. Nothing in that method ever asked whether two of the answers you invented were secretly the same answer in disguise. Time to check.

Distinguishable states

States $p, q \in Q$ are **distinguishable** if there exists a string $w \in \Sigma^*$ such that exactly one of $\hat{\delta}(p, w), \hat{\delta}(q, w)$ is in F – w is a *witness* that tells p and q apart. If no such w exists, p and q are **equivalent**, written $p \equiv q$.

The witness $w = \varepsilon$ is already a special case worth naming: if exactly one of p, q is itself in F , they are distinguishable before reading a single symbol. Every other witness has to be found by searching – except you never actually have to search, because distinguishability propagates backward one symbol at a time.



If r and s are already known to be distinguishable – by some witness w' – then p and q are distinguishable too, by the witness aw' : read a from each, land in r and s respectively, then let w' finish the job. This is the entire engine of Section 1.2: start from the $w = \varepsilon$ base case and let this one rule ripple backward through δ until it stops finding anything new.

Exercises: Spotting Distinguishability

1. In Lecture 6's ends-with-0 DFA (q_{*0} accepting, q_{*1} not), are q_{*0} and q_{*1} distinguishable? Name the shortest witness w . (Hint: you barely need the backward rule for this one – check the base case first.)
2. Suppose p, q, r are three states with $\delta(p, a) = \delta(q, a) = r$ for every symbol a , and neither p nor q is in F . Are p and q necessarily equivalent? Could they still be distinguishable by some longer witness that does *not* start with a ? (Hint: re-read the definition – it quantifies over *all* w , not just length-1 prefixes.)
3. A friend claims “ $p \equiv q$ ” and “ p, q reachable by the same set of strings” mean the same thing. Are they the same claim, or is one strictly stronger? (Hint: two states can be equivalent without a single string ever reaching both of them.)

1.2 The Table-Filling Algorithm

Turning the backward-propagation rule into something you can run to completion, rather than searching witnesses by hand, is due to E. F. Moore's 1956 paper on sequential machines – decades before Myhill-Nerode's theorem (Section 1.4) explained *why* it always terminates at the true minimum.

Method: table-filling

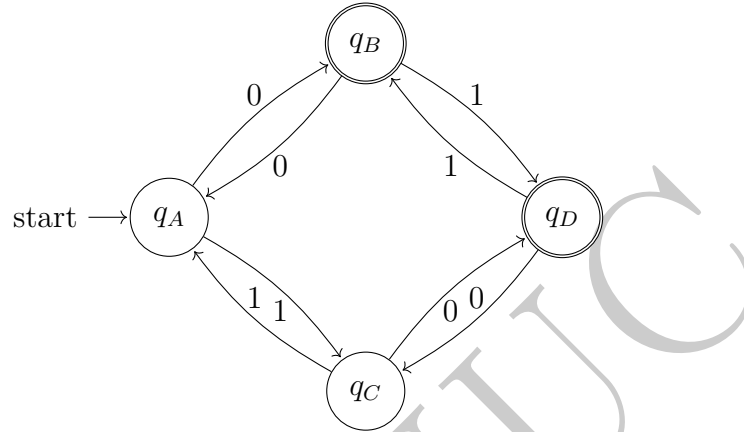
1. List every unordered pair of distinct states $\{p, q\}$ in a triangular table.
2. **Base case.** Mark $\{p, q\}$ if exactly one of p, q is in F .
3. **Propagate.** Scan every unmarked pair $\{p, q\}$: if some symbol $a \in \Sigma$ has $\{\delta(p, a), \delta(q, a)\}$ already marked, mark $\{p, q\}$ too.
4. Repeat Step 3 until a full pass marks nothing new – with only finitely many pairs and marks that never get undone, this must happen within $|Q|$ passes at worst.
5. Every pair left unmarked is equivalent. Merge each equivalence class into a single state.

To think!!

Step 4 promises the algorithm halts within $|Q|$ passes. Where exactly does that bound come from – what is the largest number of *times* a single pass can newly mark at least one pair before every pair in the whole table is already marked or provably stuck unmarked forever?

1.3 Worked Example: Minimizing a 4-State DFA

Take M over $\Sigma = \{0, 1\}$, four states arranged so that 0 and 1 each swap you between an opposite pair:



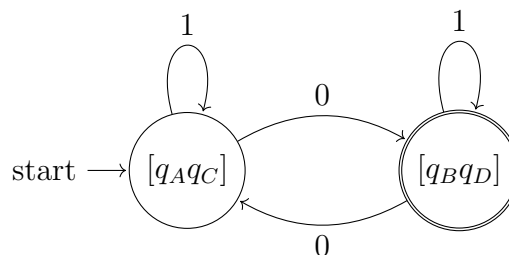
q_A is the start state; $F = \{q_B, q_D\}$. Build the triangular table. **Base case** first: every pair with exactly one state in $F = \{q_B, q_D\}$ gets marked immediately, witness $w = \varepsilon$: $\{q_A, q_B\}$, $\{q_A, q_D\}$, $\{q_C, q_B\}$, $\{q_C, q_D\}$ – four marks, one for every way to pick one state from $\{q_A, q_C\}$ and one from $\{q_B, q_D\}$.

	q_A	q_B	q_C
q_B	×		
q_C		×	
q_D	×		×

(Only the lower-left triangle is meaningful – a state compared to itself, or to a pair already checked from the other side, adds nothing new.)

Two pairs remain unmarked: $\{q_A, q_C\}$ and $\{q_B, q_D\}$. **Propagate** and check each against Section 1.1's backward rule. For $\{q_A, q_C\}$: on 0, $\delta(q_A, 0) = q_B$ and $\delta(q_C, 0) = q_D$, and $\{q_B, q_D\}$ is *not* marked, so 0 gives nothing; on 1, $\delta(q_A, 1) = q_C$ and $\delta(q_C, 1) = q_A$, which is $\{q_A, q_C\}$ itself – checking a pair against itself proves nothing. For $\{q_B, q_D\}$: on 0, $\delta(q_B, 0) = q_A$ and $\delta(q_D, 0) = q_C$, and $\{q_A, q_C\}$ is not marked either; on 1, $\delta(q_B, 1) = q_D$ and $\delta(q_D, 1) = q_B$, again the pair itself. Neither pair can mark the other, and no third pair is left to feed them a mark from outside – one more pass changes nothing, so the algorithm halts. **Final classes:** $\{q_A, q_C\}$ and $\{q_B, q_D\}$.

Merge each class into one state and carry the transitions across (every state in a class must agree on where each symbol sends it, and here they do: both q_A and q_C send 0 into $\{q_B, q_D\}$ and 1 into $\{q_A, q_C\}$):



Four states dropped to two. What language survives the cut? Every 0 crosses the $[q_Aq_C]/[q_Bq_D]$ boundary, and every 1 leaves you on whichever side you were already on – so the machine is doing nothing but counting 0’s modulo 2. M , in its bloated 4-state form, secretly recognized $L = \{w \in \{0,1\}^* : w \text{ has an odd number of 0’s}\}$ all along; the q_A/q_C and q_B/q_D split was tracking the parity of 1’s too, a fact that never once affected acceptance.

Exercises: Running the Table Yourself

1. Trace $w = 0110$ on both M (the 4-state original) and the minimized 2-state machine. Confirm they land in corresponding classes at every step, not just agree on the final answer.
2. Add a fifth state q_E to M with $\delta(q_D, 1) = q_E$ instead of q_B , $\delta(q_E, \cdot)$ a self-loop on both symbols, and $q_E \notin F$. Redo the table-filling base case with this new F . Is q_E equivalent to q_A or q_C , or is it in a class of its own? (Hint: what string distinguishes q_E from q_A ?)
3. A classmate minimizes M and gets three states instead of two, having merged q_A with q_B “because they looked similar.” Using the base case alone, explain why that merge is illegal before checking anything else.

1.4 The Myhill-Nerode Theorem

Table-filling found the equivalence classes of *one particular* DFA. That leaves a nagging question: classes of what, exactly – the machine, or the language? Two different people minimizing two different (but correct) DFAs for the same L had better end up with the same number of classes, or “the” minimal DFA would not be a meaningful phrase. The fix is to stop talking about states at all and define equivalence directly on *strings*.

The Myhill-Nerode relation $\sim_L$

Fix a language $L \subseteq \Sigma^*$. Two strings $x, y \in \Sigma^*$ are **equivalent**, written $x \sim_L y$, if for every continuation $z \in \Sigma^*$, xz and yz are either both in L or both not in L . In other words, no continuation can distinguish them.

This is exactly Section 1.1’s distinguishability, one level up: a DFA’s states p, q are equivalent precisely when *every* string reaching p is $\sim_L$ -equivalent to *every* string reaching q . Table-filling on a specific M is $\sim_L$ computed through the lens of that one machine’s states; $\sim_L$ itself does not care which machine you drew.

Myhill-Nerode Theorem

L is regular if and only if $\sim_L$ has finitely many equivalence classes. When L is regular, that number of classes equals exactly the number of states in L ’s *minimal* DFA – no correct DFA for L can ever have fewer states.

Section 1.3’s reveal was really this theorem in miniature: $\sim_L$ for “odd number of 0’s” has exactly two classes – strings with an even count of 0’s, and strings with an odd count – and

no DFA, however cleverly built, can recognize that language in fewer than two states. Every regular language has a unique minimal DFA, and that number is $\sim_L$'s class count, full stop.

A note on where this theorem comes from

John Myhill (1957) and Anil Nerode (1958) proved essentially this same characterization independently of each other within a year, both phrasing it in terms of a right-invariant equivalence relation of finite index – one more entry in automata theory's long list of results discovered twice before anyone noticed the overlap.

Nothing above stops you from running the idea in the other direction: instead of minimizing a DFA you already built, declare the $\sim_L$ -classes themselves to be the states of a new machine from the start, with $\delta([x], a) = [xa]$. That machine is automatically the minimal DFA for L , built with no wasted states to begin with – worth sitting with as a “to think” rather than a worked example here: given only L itself, could you compute enough of $\sim_L$ by hand to name its classes before ever drawing a single state?

Exercises: Thinking in Classes

1. For $L = \{w \in \{0,1\}^* : w \text{ ends with } 1\}$, name the $\sim_L$ -classes directly, without building a DFA first. How many are there, and does that match the state count of the DFA you would have built by Lecture 6's method for this language?
2. Are ε and 0 $\sim_L$ -equivalent for $L = \{w : w \text{ has an odd number of } 0\text{'s}\}$, Section 1.3's language? Name a continuation z that settles it, then check your answer against which class each string reaches in the minimized machine.
3. If $\sim_L$ has infinitely many classes for some L , what does the Myhill-Nerode theorem let you conclude about L immediately, without attempting any DFA construction at all?

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (HMU) §4.4.1 “Testing Equivalence of States” – the table-filling algorithm matching Section 1.2, stated with the same base-case/propagate structure.
2. (HMU) §4.4.3 “Minimization of DFA's” – building the minimized machine from the equivalence classes, matching Section 1.3's merge step.

3. (HMU) §4.4.4 “Why the Minimized DFA Can’t Be Beaten” – HMU’s own version of the uniqueness argument behind Section 1.4’s closing corollary.
4. (SIPSER) Problems 1.51–1.52 (end of Chapter 1) – Sipser’s own guided derivation of the Myhill-Nerode relation and theorem, given as an exercise rather than a numbered section; problem numbers may shift slightly across printings of the 3rd edition, so check your own copy.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 Regular Expressions, Automata, and the Pumping Lemma

Lecture 3 gave regular expressions as a compact linear notation for exactly the languages Lecture 2's algebra already knew how to build – union, concatenation, star. Lectures 6 through 9 gave DFAs, NFAs, and ε -NFAs as machines for the same job. Two notations for the same idea beg to be connected mechanically, not just by argument: given a regular expression, build the automaton, with no cleverness required at any step. That is the first half of this document. The second half turns the question around: every regular expression describes a regular language, but not every language is regular – and we need a way to *prove* that a given language is not, since no amount of failing to find a DFA for it is itself a proof. The pumping lemma is that tool, and we put it to work on five languages that no automaton, however large, can ever recognize.

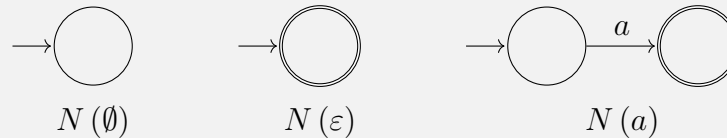
1.1 From Regex to ε -NFA: Thompson's Construction

Look again at how Lecture 3 built $L(R)$ recursively – a base case per symbol, $\emptyset$ and ε ; an inductive case per operator, $|$, concatenation, $*$ – and how Lecture 7 built ε -NFAs by gluing small, obviously-correct fragments together with free moves. The two recursions are the *same* recursion. Build one tiny ε -NFA fragment per base case, one gluing rule per operator, and structural induction on the regular expression's own definition builds the whole machine for you. This is Thompson's construction, published by Ken Thompson – already met in Lecture 3 through `grep` – in his 1968 *CACM* paper *Regular Expression Search Algorithm*, as a compilation step for exactly the text-editor matching that paper was about.

Method: Thompson's construction

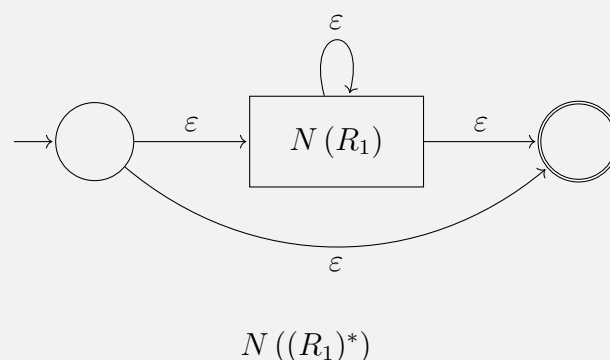
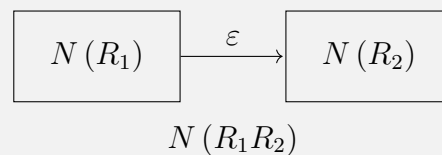
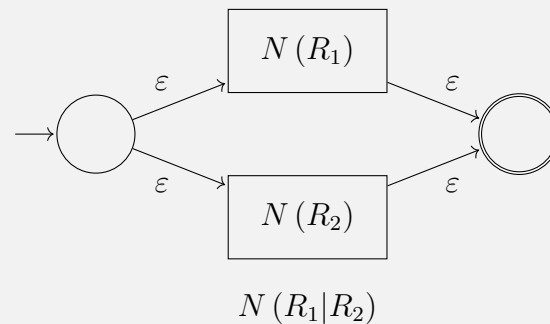
Write $N(R)$ for the ε -NFA fragment built for regular expression R , with exactly one start state and one accepting state.

Base cases.



$N(\emptyset)$ has no transitions at all and a non-accepting start – nothing is ever accepted. $N(\varepsilon)$'s single state is both start and accept, with no transitions – only ε is accepted, since any real symbol has nowhere to go.

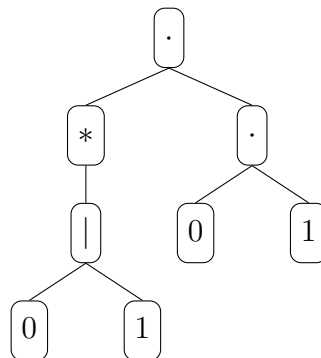
Inductive cases. Given fragments $N(R_1)$ and $N(R_2)$ already built:



In every case, R_1 's and R_2 's own old accepting states stop being accepting – they are demoted to ordinary states once their job is to hand control onward via ε , and exactly one new accepting state is designated per rule.

Apply the recursion to $(0|1)^*01$ – Lecture 7 and Lecture 8's running ends-with-01 language, built by hand there and now built mechanically. The regular expression's own struc-

ture is the recipe:



Read the tree bottom-up, tallying states as each rule fires: $N(0)$ and $N(1)$ are 2 states each. $N(0|1)$ glues them with 2 new states – 6 so far. $N((0|1)^*)$ adds 2 more new states around that – 8. The tail $N(0)$ and $N(1)$ contribute 2 states each, concatenated with no new states of their own – 4 more. The final concatenation of $N((0|1)^*)$ with $N(01)$ adds one ε -edge and no new states. **Total: 12 states** – against the 3 states Lecture 7 built by hand, and the 2 states Lecture 9 would minimize either one down to. Thompson’s construction is never wrong, but it is never economical either; that gap is exactly why Lecture 9’s minimization earns its keep on machine-generated automata, not just hand-built ones.

Exercises: Building by Hand

1. Draw $N(a^*b)$ in full, state by state, using the boxed rules directly – no shortcuts. How many states and how many ε -edges does it have?
2. For $(0|1)^*01$ above, which single state, if you traced a run of the machine, would you expect to see visited most often for a long input? (Hint: think about which fragment the star wraps.)
3. Lecture 7 gave an algorithm for eliminating ε -transitions from a hand-built ε -NFA. Apply that same elimination algorithm to $N(a^*b)$ from Exercise 1 above, and compare the resulting raw NFA’s state count against a hand-built NFA for the same language, $\{a^n b : n \geq 0\}$.

1.2 The Pumping Lemma for Regular Languages

Every construction so far in this course proves a language regular by *exhibiting* a machine for it. That only ever runs in one direction. To show a language is *not* regular, exhibiting nothing proves nothing – failing to find a DFA is not the same as showing none exists. What is needed is a property every regular language is *forced* to have, so that finding one language without that property settles the matter outright.

The Pumping Lemma for Regular Languages

If L is regular, there exists a **pumping length** $p \geq 1$ such that every $w \in L$ with $|w| \geq p$ can be written $w = xyz$ with:

1. $|xy| \leq p$,
2. $|y| \geq 1$,
3. $xy^iz \in L$ for every $i \geq 0$.

The proof is a single pigeonhole observation: take a DFA for L with p states, and run it on any $w \in L$ with $|w| \geq p$. Reading the first p symbols visits $p + 1$ states – more states than the machine has – so some state repeats. Everything between the two visits to that repeated state is a loop, y , that can be cut out entirely ($i = 0$) or repeated any number of times ($i \geq 2$) without changing where the run ends up, since it always returns to the very same state it started the loop from.

Reading the lemma as a two-player game makes it usable: *you* do not get to pick w , x , y , or z freely and call it a proof – an adversary, standing in for “some DFA with p states,” picks the split xyz (subject to $|xy| \leq p$, $|y| \geq 1$) after seeing your w , and you only need to defeat *every* split the adversary could possibly choose by finding, for each one, some i that breaks $xy^iz \in L$. If you can always win this game for a language L , L cannot be regular – because if it were, the adversary’s promised split would have to survive every i , and it never does.

To think!!

The pumping lemma is a *necessary* condition for regularity, not a sufficient one. Suppose a language L happens to satisfy the pumping property (some p exists, every long string splits and pumps correctly) purely by accident. Does the lemma let you conclude L is regular? What would you need instead to prove regularity, going by everything else in this course?

1.3 Five Languages That Are Not Regular

Each proof below follows the same shape: pick a witness string $w \in L$ with $|w| \geq p$, show every legal split forces y into a predictable shape, then find an i that breaks membership.

$L_1 = \{0^n 1^n : n \geq 0\}$. Choose $w = 0^p 1^p$. Since $|xy| \leq p$, xy sits entirely inside the leading 0^p block, so $y = 0^k$ for some $1 \leq k \leq p$. Pump $i = 2$: $xy^2z = 0^{p+k} 1^p$ has strictly more 0’s than 1’s, so $xy^2z \notin L_1$ for every valid k . Not regular.

$L_2 = \{ww : w \in \{0,1\}^*\}$. Choose $w_0 = 0^p 10^p 1$ (this is ww for $w = 0^p 1$). Again $y = 0^k$, $1 \leq k \leq p$, from the leading block. Pump $i = 0$: $xz = 0^{p-k} 10^p 1$. If this were uu , the two 1’s could not both sit in the same copy of u (the other copy, being identical, would then need two 1’s as well, overshooting the total of two) – so each copy contains exactly one 1. But the second copy’s 1 is necessarily the very last symbol of the whole string, while the first copy’s

1 is followed by more symbols before that copy ends, unless $k = 0$. Since $k \geq 1$ is forced, the two copies can never agree, and $xz \notin L_2$. Not regular.

$L_3 = \{w \in \{0,1\}^* : w = w^R\}$ (**all palindromes**). Choose $w = 0^p 10^p$, itself a palindrome of length $2p + 1$. As before $y = 0^k$, $1 \leq k \leq p$, from the leading block only. Pump $i = 0$: $xz = 0^{p-k} 10^p$, whose reverse is $0^p 10^{p-k}$. These agree only if $p - k = p$, i.e. $k = 0$ – impossible, since $k \geq 1$. So $xz \neq (xz)^R$, and $xz \notin L_3$. Not regular.

$L_4 = \{0^i 1^j : i > j \geq 0\}$. Choose $w = 0^{p+1} 1^p$ ($i = p + 1 > p = j$). The first p symbols of w are all 0's, so $y = 0^k$, $1 \leq k \leq p$, as always. Pump $i = 0$: $xz = 0^{p+1-k} 1^p$. Since $1 \leq k \leq p$, the new count of 0's satisfies $p + 1 - k \leq p$, no longer strictly more than the p 1's – the defining inequality $i > j$ fails. Not regular.

$L_5 = \{0^{n^2} : n \geq 0\}$ (**perfect-square lengths**). Choose $w = 0^{p^2}$. Every symbol is 0, so $y = 0^k$ for some $1 \leq k \leq p$, regardless of exactly where x, y, z split. This time pump up , $i = 2$: $xy^2z = 0^{p^2+k}$. Since $1 \leq k \leq p$, $p^2 < p^2 + k \leq p^2 + p < p^2 + 2p + 1 = (p + 1)^2$ – so $p^2 + k$ lies strictly between the consecutive perfect squares p^2 and $(p + 1)^2$, and is therefore not itself a perfect square, for every $p \geq 1$. Not regular.

A note on where this proof technique comes from

The pumping lemma for *regular* languages is the easy, folklore-level pigeonhole argument stated above. Its far more famous relative, the pumping lemma for *context-free* languages, was proved formally by Bar-Hillel, Perles, and Shamir in 1961 – the regular-language version is usually taught first precisely because it is the simpler special case of essentially the same idea, with one repeated state standing in for one repeated pair of grammar symbols.

Exercises: Proving Non-Regularity

1. Prove $L = \{0^n 1^m : n \neq m\}$ is not regular. (Hint: this is *not* simply L_4 with the inequality flipped – L also contains every $0^i 1^j$ with $i < j$. Does your witness string from L_4 's proof still work here?)
2. Prove $L = \{a^i b^j c^k : i = j \text{ or } j = k\}$ is not regular, using L_1 's proof as a template. (Hint: choose a witness where pumping breaks *both* disjuncts at once.)
3. A classmate “proves” $\{0^n 1^n\}$ regular by exhibiting a DFA that correctly accepts $0^n 1^n$ for every n from 0 to 100. Using this document's opening paragraph, explain precisely what has and has not been shown.

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.

2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (HMU) §3.2.3 “Converting Regular Expressions to Automata” – Thompson’s construction matching Section 1.1, in the same base-case/inductive-case structure.
2. (SIPSER) §1.3 “Regular Expressions,” specifically the proof of Lemma 1.55 – Sipser’s own version of the same construction, building the NFA fragments in the opposite order (concatenation before union) but with identical fragments.
3. (HMU) §4.1.1 “The Pumping Lemma for Regular Languages” and §4.1.2 “Applications of the Pumping Lemma” – the formal statement and worked examples matching Sections 1.2-1.3.
4. (SIPSER) §1.4 “Nonregular Languages” – Sipser’s own pumping lemma statement (phrased as the same adversarial game) and non-regularity proofs, including $\{0^n1^n\}$ and a version of the perfect-square example.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.